EXPERIMENT 8

Name-Spandan Deb
Class-D20A
Roll no-20

Aim-Implement the Blockchain platform Ganache.

Theory-
What is Ganache?
Ganache is a local blockchain simulator used for developing and testing Ethereum smart contracts.

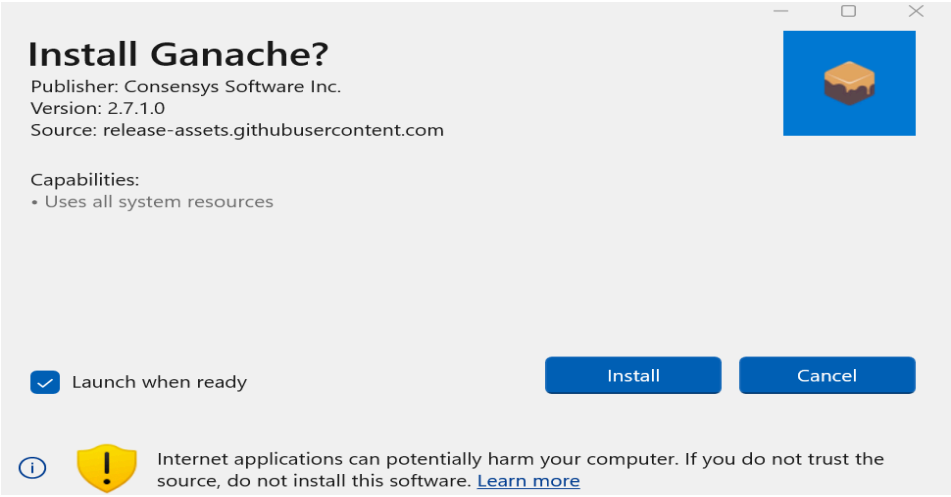
- It creates a personal blockchain on your computer
- Provides pre-funded accounts with test ETH
- Allows instant transactions (no real gas cost)
- Used for testing before deploying on real networks

List the steps involved in connecting Ganache environment with a Metamask and Remix IDE.

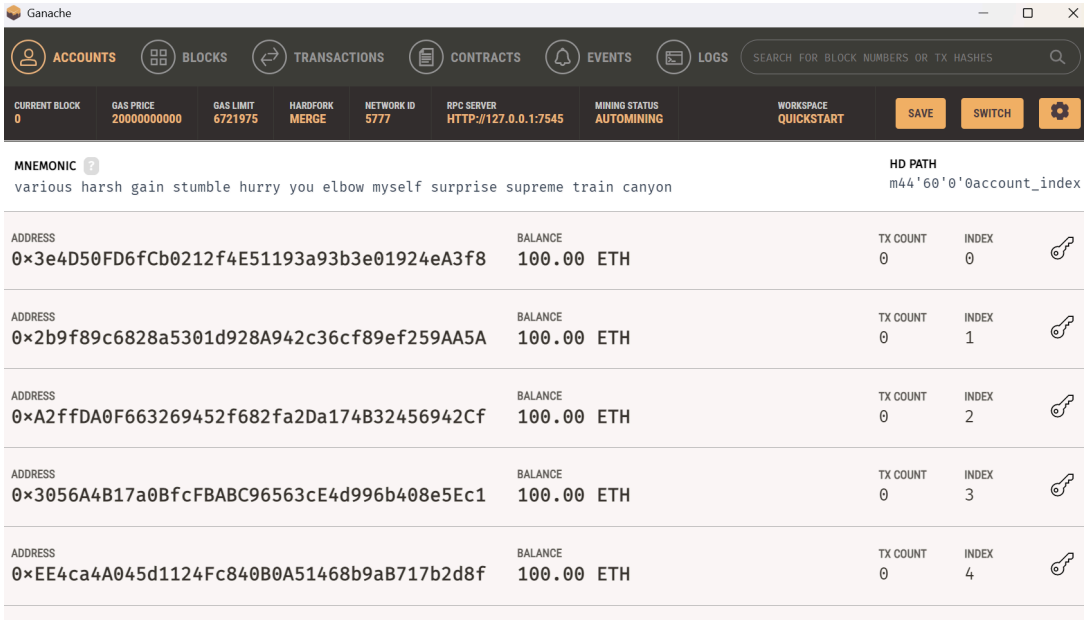
1. Start Ganache and note RPC URL and Chain ID.
2. Open MetaMask.
3. Add a new network in MetaMask using Ganache RPC URL and Chain ID.
4. Copy a private key from Ganache.
5. Import the account into MetaMask using the private key.
6. Open Remix IDE.
7. Go to “Deploy & Run Transactions”.
8. Select environment as Injected Provider - MetaMask.
9. Connect MetaMask to Remix.
10. Compile the Solidity contract.
11. Deploy the contract and confirm the transaction in MetaMask.
12. Interact with the contract functions to perform transactions.

Implementation-

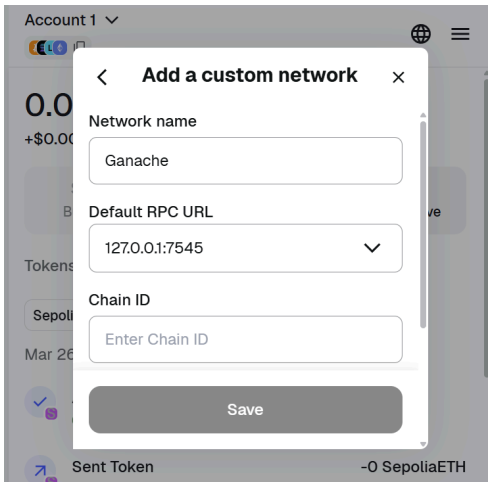
Installing Ganache



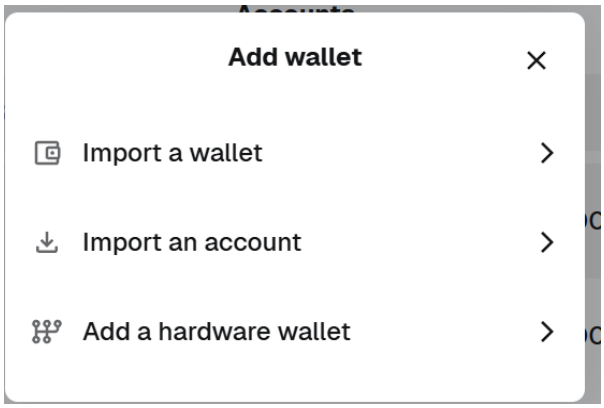
After installing click on quickstart ethereum to get 10 accounts worth 100 ETH for free.



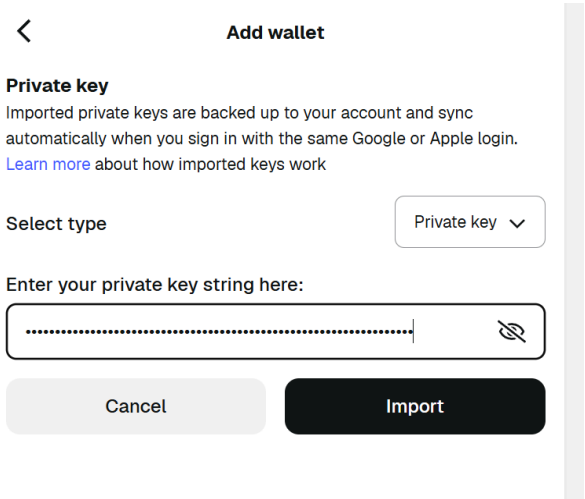
Add ganache network in metamask



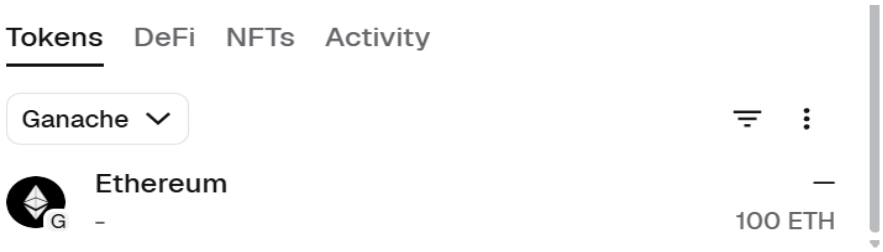
Select import an account in add wallet



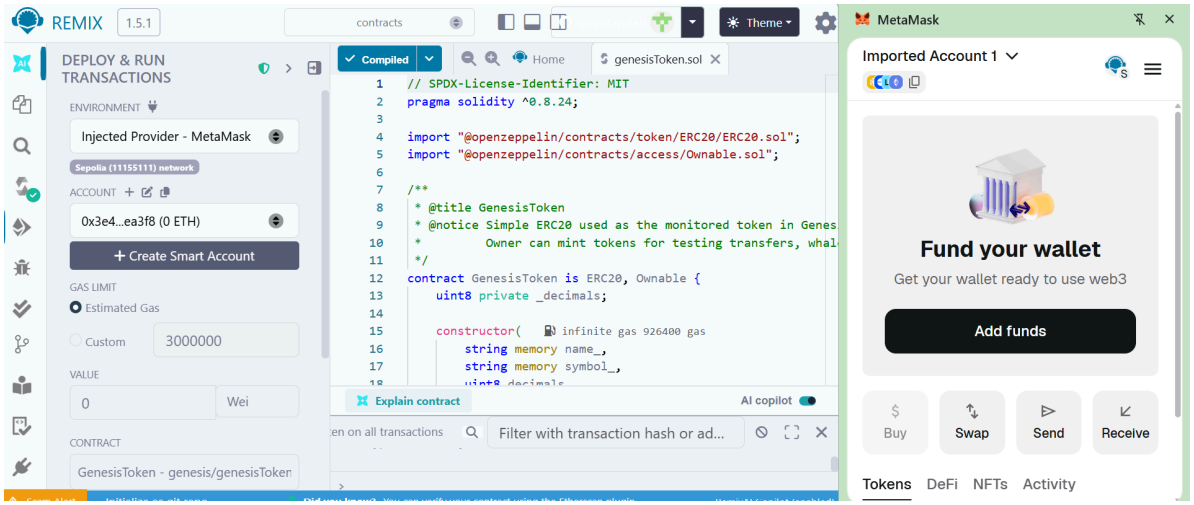
Copy the private key of any of the 10 accounts and paste it here



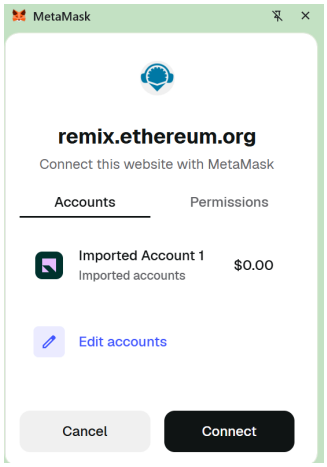
Account imported successfully balance is 100 ETH



Select in environment injected provider metamask



Connect with remix ide



GenesisToken.sol

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.24;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

import "@openzeppelin/contracts/access/Ownable.sol";

/**

 * @title GenesisToken

 * @notice Simple ERC20 used as the monitored token in Genesis demos.

 *         Owner can mint tokens for testing transfers, whale movements, etc.

 */

contract GenesisToken is ERC20, Ownable {

    uint8 private _decimals;


    constructor(

        string memory name_,

        string memory symbol_,

        uint8 decimals_,

        uint256 initialSupply

    ) ERC20(name_, symbol_) Ownable(msg.sender) {

        _decimals = decimals_;

        _mint(msg.sender, initialSupply * (10 ** decimals_));

    }
```

```
    function decimals() public view override returns (uint8) {
        return _decimals;
    }

    function mint(address to, uint256 amount) external onlyOwner {
        _mint(to, amount);
    }

    function burn(uint256 amount) external {
        _burn(msg.sender, amount);
    }
}
```

GenesisVault.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;
import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
/**
 * @title GenesisVault
 * @notice On-chain vault that emits rich events for every deposit, withdrawal,
 *         and internal transfer.  Genesis monitors these events in real time.
```

```
 *
 *       KEY BLOCKCHAIN FEATURES SHOWCASED:
 *         1. Event-driven architecture (Solidity `event`)
 *         2. Inbound/outbound tracking on-chain
 *         3. Reentrancy protection
 *         4. Per-user balance accounting
 *         5. Emergency pause (circuit-breaker pattern)
 */
contract GenesisVault is Ownable, ReentrancyGuard {
    using SafeERC20 for IERC20;

    // ═══════════════════════════════ State ═══════════════════════════════
    IERC20 public immutable token;

    mapping(address => uint256) public balances;
    uint256 public totalDeposits;
    uint256 public totalWithdrawals;
    bool public paused;

    // ═══════════════════════════════ Events (the core of Genesis monitoring) ═══════════
    /// @notice Emitted on every deposit — Genesis indexes this immediately.
    event Deposit(
        address indexed user,
```

```
    uint256 amount,
    uint256 newBalance,
    uint256 timestamp
);

/// @notice Emitted on every withdrawal — potential "outbound whale" signal.
event Withdrawal(
    address indexed user,
    uint256 amount,
    uint256 remainingBalance,
    uint256 timestamp
);

/// @notice Vault-internal transfer between two users.
event InternalTransfer(
    address indexed from,
    address indexed to,
    uint256 amount,
    uint256 timestamp
);

/// @notice Large movement detected on-chain — threshold exceeded.
event LargeMovement(
```

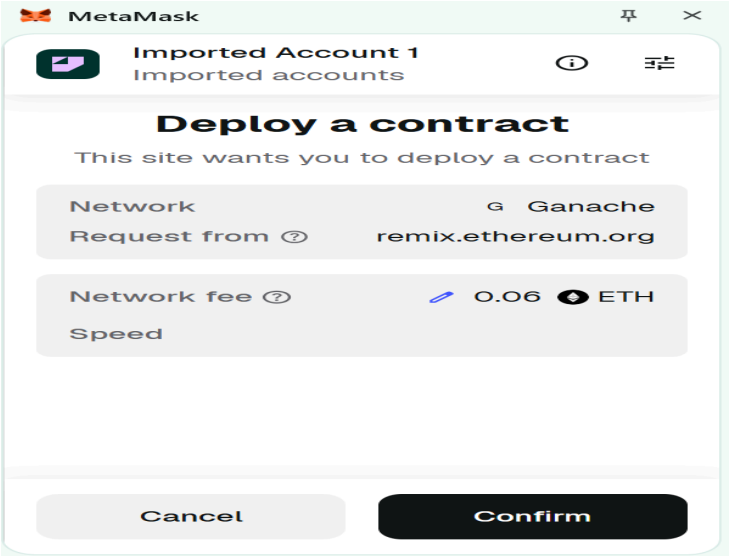
```
    address indexed user,
    string movementType,  // "deposit" | "withdrawal"
    uint256 amount,
    uint256 thresholdUsed,
    uint256 timestamp
);

/// @notice Emergency pause toggled.
event EmergencyAction(
    string action,   // "paused" | "unpaused"
    address indexed triggeredBy,
    uint256 timestamp
);
// ──────────────────────────────── Modifiers ────────────────────────────
modifier whenNotPaused() {
    require(!paused, "Vault: paused");
    _;
}
// ──────────────────────────────── Constructor ──────────────────────────
constructor(address tokenAddress) Ownable(msg.sender) {
    token = IERC20(tokenAddress);
}
```

```
// ──────────────────────────────── Core functions ────────────────────────
/**
 * @notice Deposit tokens into the vault.
 * @param amount Number of token units (with decimals).
 */
function deposit(uint256 amount) external nonReentrant whenNotPaused {
    require(amount > 0, "Vault: zero amount");
    token.safeTransferFrom(msg.sender, address(this), amount);
    balances[msg.sender] += amount;
    totalDeposits += amount;
    emit Deposit(msg.sender, amount, balances[msg.sender], block.timestamp);
    // On-chain large-movement detection (threshold hard-coded for demo;
    // the ThresholdEngine contract provides user-customisable thresholds).
    if (amount >= 100_000 * 1e6) {
        emit LargeMovement(msg.sender, "deposit", amount, 100_000 * 1e6, block.timestamp);
    }
}
/**
 * @notice Withdraw tokens from the vault.
 */
function withdraw(uint256 amount) external nonReentrant whenNotPaused {
    require(amount > 0, "Vault: zero amount");
    require(balances[msg.sender] >= amount, "Vault: insufficient balance");
```

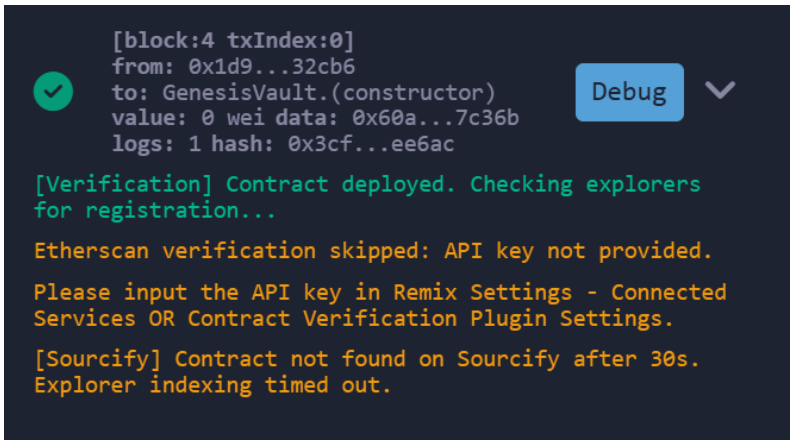
```
        balances[msg.sender] -= amount;

        totalWithdrawals += amount;

        token.safeTransfer(msg.sender, amount);

        emit Withdrawal(msg.sender, amount, balances[msg.sender], block.timestamp);

        if (amount >= 100_000 * 1e6) {

            emit LargeMovement(msg.sender, "withdrawal", amount, 100_000 * 1e6,
block.timestamp);

        }

    }

    /**

     * @notice Transfer balance to another vault user without touching the token contract.

     *         Demonstrates internal accounting — useful for Layer-2 patterns.

     */

    function internalTransfer(address to, uint256 amount) external nonReentrant whenNotPaused
{

        require(to != address(0) && to != msg.sender, "Vault: invalid recipient");

        require(balances[msg.sender] >= amount, "Vault: insufficient balance");

        balances[msg.sender] -= amount;

        balances[to] += amount;

        emit InternalTransfer(msg.sender, to, amount, block.timestamp);

    }

    // ──────────────────────────────── Admin / Emergency ──────────────
```

```
    function pause() external onlyOwner {
        paused = true;
        emit EmergencyAction("paused", msg.sender, block.timestamp);
    }
    function unpause() external onlyOwner {
        paused = false;
        emit EmergencyAction("unpaused", msg.sender, block.timestamp);
    }
    // ──────────────────────────────── View helpers ────────────────────────
    function getVaultStats() external view returns (
        uint256 _totalDeposits,
        uint256 _totalWithdrawals,
        uint256 _vaultBalance,
        bool _paused
    ) {
        return (totalDeposits, totalWithdrawals, token.balanceOf(address(this)), paused);
    }
}
```

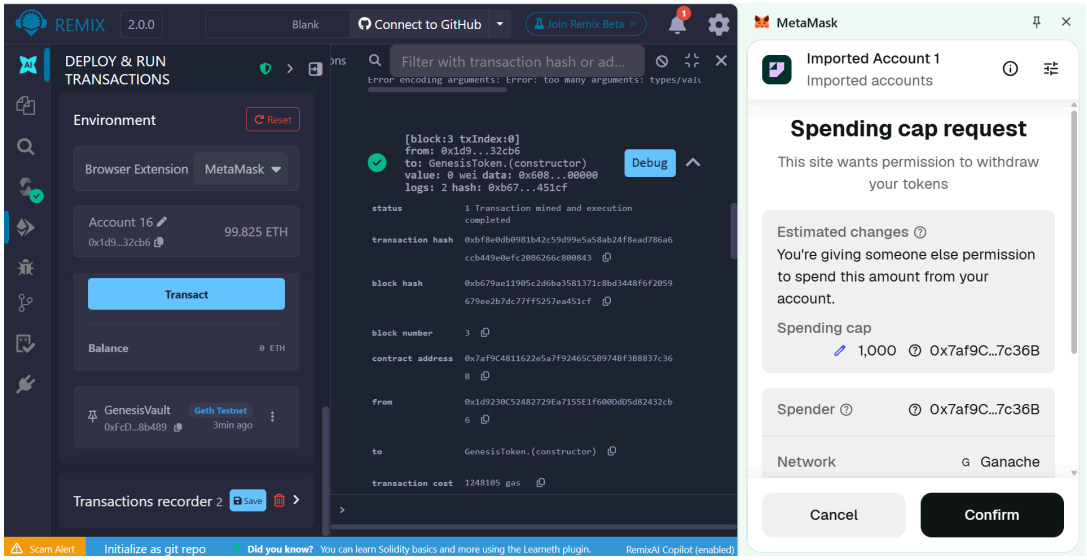
Contract Deployment of GenesisToken.sol



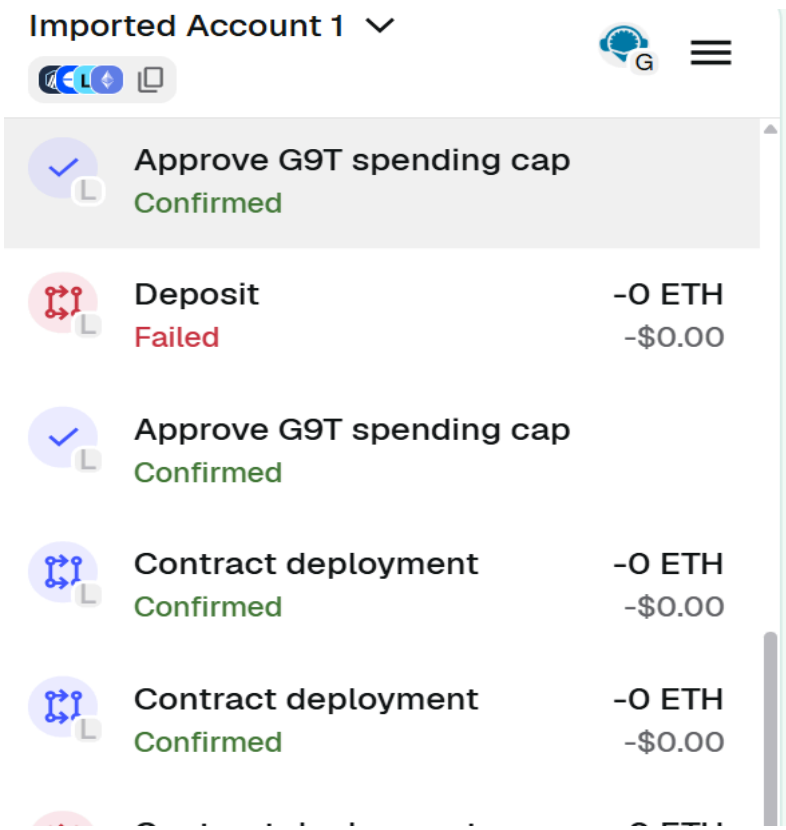
Also Deployed GenesisVault.sol



Give vault the permission to move tokens by using approve function in GenesisToken.sol



Deposit the tokens into the vault



Conclusion-

Ganache is a local Ethereum blockchain used for testing smart contracts. By connecting Ganache to MetaMask and then linking MetaMask with Remix IDE, developers can deploy and execute smart contracts in a simulated environment without using real cryptocurrency, enabling safe and efficient testing of blockchain applications.